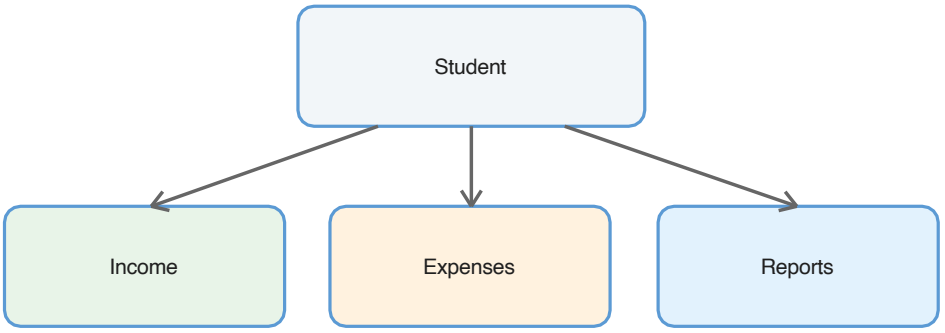
STUDENT MONEY TRACKER

A Beginner-Friendly Python Project for Personal Expense Management



Project Type	Python Application
Technology	Python 3.14.7
Storage	Locally Stored
Institution	VIT Bhopal University
Academic Year	2026–27
Student Name	PRAKHAR PUNIIT
Registration No.	26BAI10202

1. Introduction

Managing money is an important practical skill for students. College students commonly receive money from parents, scholarships, part-time work or other sources and spend it on food, travel, subscriptions, stationery, entertainment and other needs. Without a simple record, it becomes difficult to understand where the money is going.

The Student Money Tracker is a beginner-friendly Python console application designed to help a student record income and expenses, classify transactions, calculate the available balance and view a basic spending summary. The project applies fundamental programming concepts such as variables, input/output, conditional statements, loops, functions, lists/dictionaries and file handling.

The application is intentionally kept simple so that a beginner can understand the complete flow from user input to calculation and reporting. It can later be extended into a graphical or web application without changing the basic financial logic.

Objectives

• Provide a simple way to record income and expenses.

• Maintain a running balance from recorded transactions.

• Organize expenses using categories such as Food, Travel, Education and Entertainment.

• Allow the user to review stored transactions and a spending summary.

• Reduce manual calculation and make personal budgeting easier to understand.

2. Problem Statement

Students often track expenses informally through memory, chat messages or scattered notes. This can lead to forgotten transactions, incorrect totals and poor awareness of monthly spending. A lightweight money-tracking application can solve this by providing one place to enter, calculate and review transactions.

Problem statement: Design and implement a simple Python-based money tracker that allows a student to record financial transactions, distinguish income from expenses, calculate the current balance and generate a meaningful summary of spending while validating user input.

Scope

• Single-user student budgeting.

• Manual entry of transactions.

• Basic category-based expense analysis.

• Local storage rather than online banking integration.

• No real-money transactions, payments or bank-account access.

3. Functional Requirements

ID	Requirement	Description
1	Add Income	Record an income amount with date/source information.

ID	Requirement	Description
2	Add Expense	Record an expense amount and category.
3	View Transactions	Display previously entered transactions.
4	Calculate Balance	Calculate total income minus total expenses.
5	Category Summary	Show spending totals by category.
6	Input Validation	Reject invalid amounts, empty fields and invalid menu choices.
7	Persistent Storage	Save transaction data locally so it can be reused later.
8	Exit Safely	Close the program without losing already saved data.

The three major functional modules required by the project guidelines are: Transaction Management, Balance & Summary Processing, and Data Storage/Validation.

4. Non-functional Requirements

Requirement	Target for the project
Usability	Menu-driven interface with clear prompts and readable results.
Performance	Normal operations should complete immediately for a student-sized dataset.
Reliability	Invalid input should not terminate the application unexpectedly.
Maintainability	Functions should have focused responsibilities and meaningful names.
Data Integrity	Amounts must be numeric and expenses/income should be stored consistently.
Resource Efficiency	The application should run on a basic student laptop with minimal memory.

5. System Architecture

The project follows a simple layered architecture. The user interacts with the console interface. The input/validation layer converts and checks user input. The transaction layer creates and manages transaction records. The calculation/reporting layer derives totals and summaries. The storage layer saves and loads the records from a local file.

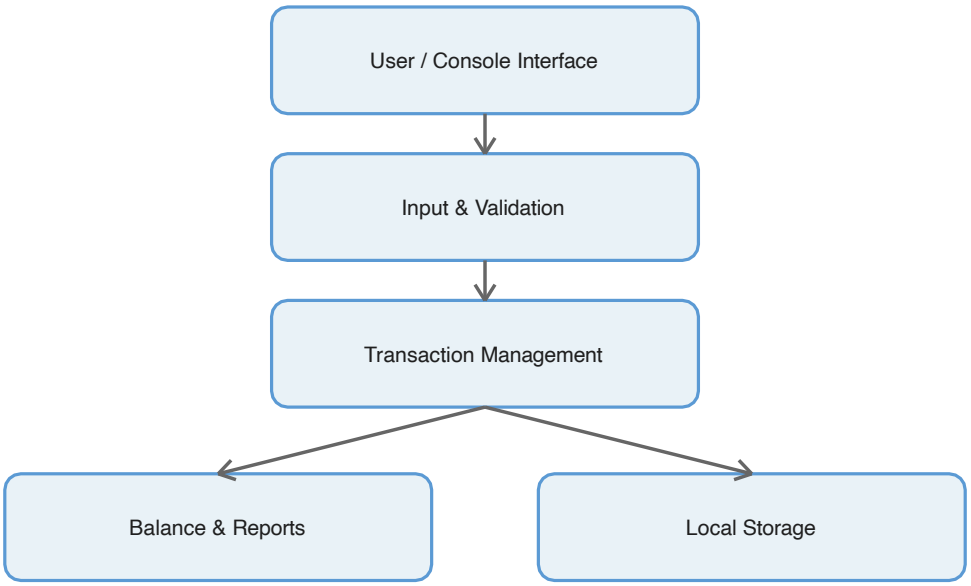


Figure 1. High-level system architecture

6. Design Diagrams

6.1.Use Case Diagram

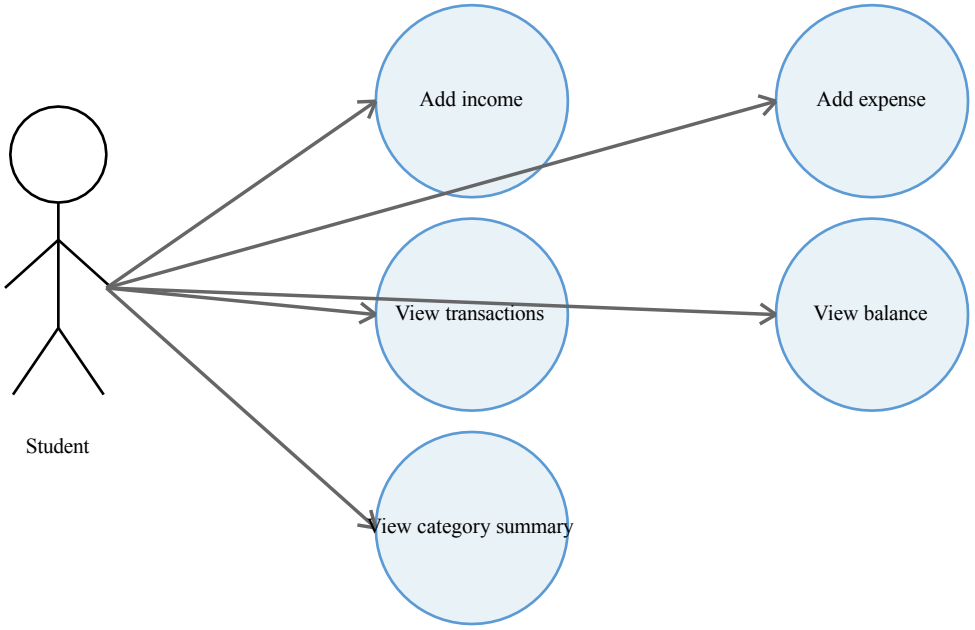


Figure 2. Use cases of the Student Money Tracker

6.2.Workflow Diagram

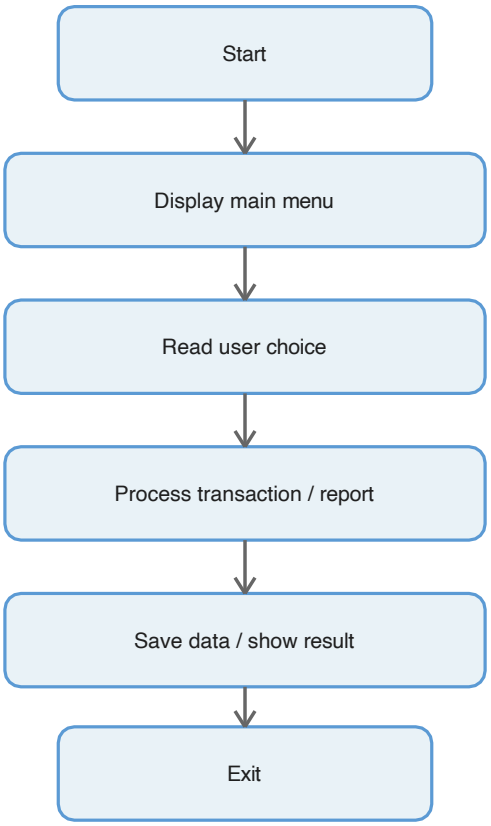


Figure 3. Simplified workflow

6.3.Sequence Diagram

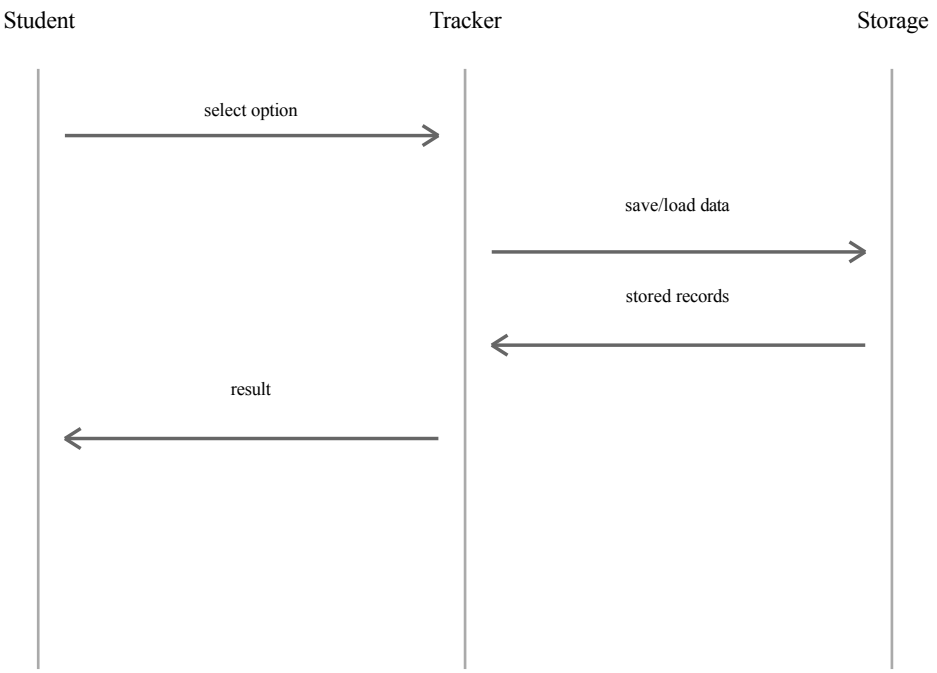
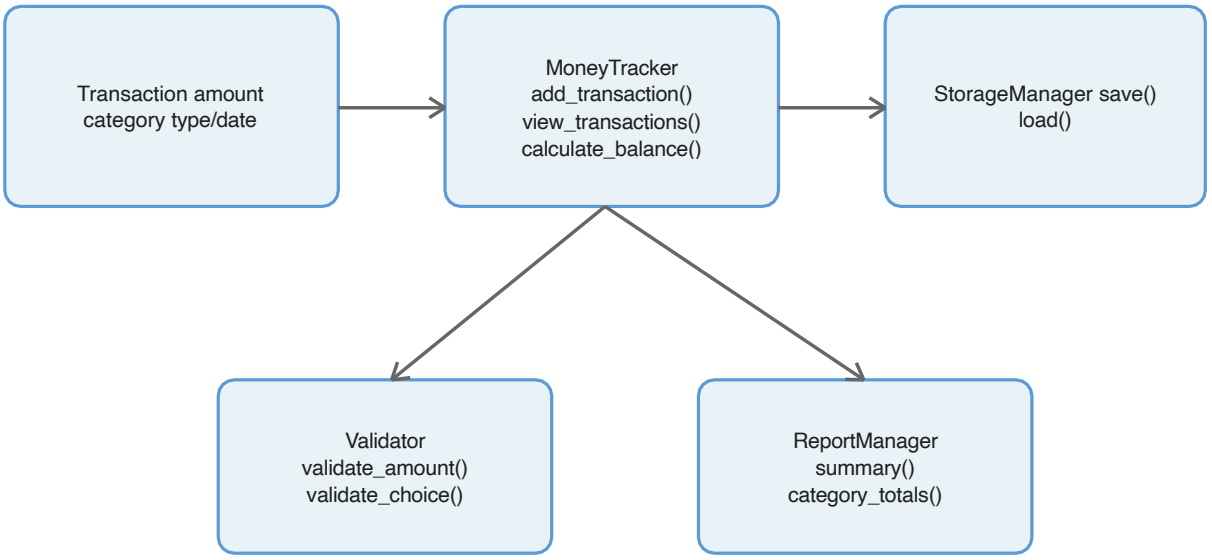


Figure 4. Sequence for a typical transaction operation

6.4.Class / Component Diagram



6.5.ER Diagram

Figure 5. Logical component structure

Because the project uses simple local storage rather than a relational database, a formal database is not required. For documentation purposes, the logical data model can be represented as follows:

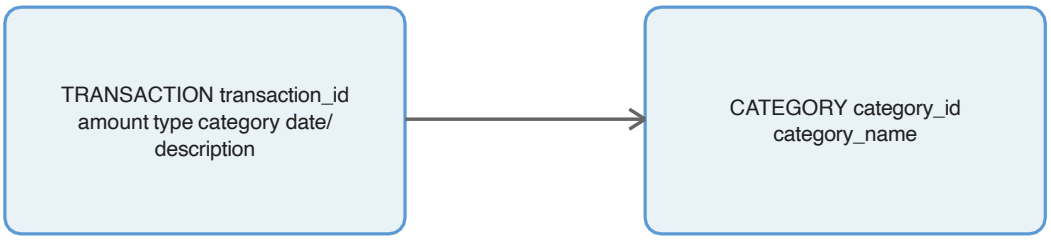


Figure 6. Logical ER-style representation (if converted to a database)

7. Design Decisions & Rationale

Decision	Rationale
Python console application	Appropriate for a beginner project and directly demonstrates programming fundamenta
Menu-driven interaction	Easy for a first-time user to understand and simple to implement.
Functions for modules	Separates input, transaction processing, calculations and storage logic.
Local file storage	Keeps the project lightweight and avoids requiring a server/database setup.
Category-based expenses	Makes the tracker more useful than simply storing a total.
Validation before processing	Prevents common errors such as negative amounts or invalid menu choices.

ls.

8. Implementation Details

The application can be implemented as a small Python project with the following logical files/modules. If the final submission uses a single Python file, these components can remain as separate functions within that file.

Module	Responsibility
main.py	Main menu and program loop.
transactions.py	Create and manage income/expense records.
validation.py	Validate numeric values, categories and choices.
reports.py	Calculate balance and category-wise totals.
storage.py	Read and write transaction records.
data/transactions.csv	Local transaction data file.

Core processing logic

```
total_income = sum(t["amount"] for t in transactions if t["type"] == "income")
total_expense = sum(t["amount"] for t in transactions if t["type"] == "expense")
balance = total_income - total_expense

for transaction in transactions:
if transaction["type"] == "expense":
category_totals[transaction["category"]] += transaction["amount"]
```

The exact implementation may use lists/dictionaries or CSV rows depending on the final version submitted to GitHub. The important design principle is that the same calculation logic is reused rather than manually calculating totals in multiple menu options.

9. Results

The following representative console output demonstrates the expected result of the application. Actual screenshots from the final GitHub version can be inserted here before submission.

```
========================================
STUDENT MONEY TRACKER
========================================
1. Add Income
2. Add Expense
3. View Transactions
4. View Balance
5. Expense Summary
6. Exit

Enter choice: 2
Enter amount: 250
Enter category: Food
Expense added successfully.

Current balance: Rs. 4,750

Expense Summary
Food Rs. 1,250
Travel Rs. 600
Education Rs. 900
Entertainment Rs. 500
--------------------------------------------------
Total Expense: Rs. 3,250
```

Expected outcome: the program accepts the transaction, stores it, updates the balance and reflects the expense in the appropriate category summary.

Suggested screenshots for the final report

• Main menu screen.

• Adding an income transaction.

• Adding an expense transaction with category.

• Transaction list.

• Balance and expense summary.

• Validation message for invalid input.

10. Testing Approach

Testing focuses on normal inputs, boundary values and invalid inputs. Each functional module should be checked independently and then tested through the complete menu workflow.

Test ID	Input / Action	Expected Result
T01	Add income: 5000	Income stored; balance increases by 5000.
T02	Add expense: 250, Food	Expense stored; balance decreases by 250.

T03	Enter letters instead of amount	Validation message; program continues.
T04	Enter negative expense	Input rejected or requested again.
T05	View transactions	All saved transactions displayed.
T06	View balance	Income – expense calculated correctly.
T07	Category summary	Expense totals grouped by category.
T08	Exit program	Program terminates normally.

11. Challenges Faced

- Designing the program so that it remains simple enough for a beginner while still containing multiple functional modules.
- Handling invalid user input without crashing the program.
- Keeping income and expense calculations consistent.
- Designing a storage format that is easy to read and modify.
- Making the category summary useful without introducing unnecessarily complex libraries.
- Structuring the project so that it can later be expanded into a GUI or web application.

12. Learnings & Key Takeaways

- How to convert a real-world problem into functional and non-functional requirements.
- How functions can divide a program into manageable components.
- How loops and conditional statements control application flow.
- How lists and dictionaries can represent structured transaction data.
- How file handling can provide basic persistence.
- Why input validation is important for reliability.
- How diagrams such as use-case, workflow and sequence diagrams communicate software design.
- How GitHub and documentation support reproducible project development.

13. Future Enhancements

Enhancement	Possible implementation
Graphical User Interface	Build a Tkinter desktop interface.
Database	Replace file storage with SQLite.
Monthly budgets	Allow a user to set category-wise spending limits.
Charts	Add pie/bar charts for category and monthly spending.
Search & filters	Filter transactions by date, category or type.
Authentication	Add local user accounts for multiple users.

Cloud backup	Synchronize data through a secure backend.
Export	Export monthly reports to CSV or PDF.

14. References

• Python Software Foundation. Python 3 Documentation — https://docs.python.org/3/

• Python Software Foundation. csv module documentation — https://docs.python.org/3/library/csv.html

• Python Software Foundation. json module documentation — https://docs.python.org/3/library/json.html

• GitHub Docs. About repositories and GitHub basics — https://docs.github.com/

• VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines (course-provided project brief).

15. Conclusion

The Student Money Tracker demonstrates how a common student problem can be converted into a small, structured software solution. The project covers requirements analysis, modular design, input validation, transaction processing, basic storage, reporting and testing. Its simple architecture makes it suitable as a beginner Python project while leaving a clear path for future extensions such as a GUI, database and visual analytics.

End of Project Report